

Documentation du script : openstack-migrate.sh

Introduction

Ce script permet la migration de machines virtuelles (VM) sous OpenStack d'un hôte source vers un hôte cible dans un environnement Grid'5000. Il intègre plusieurs vérifications pour s'assurer de la compatibilité entre les clusters, les configurations Xen, les processeurs et les règles définies dans un fichier JSON. Comme pour les autres fichiers il est important de lire ce fichier après avoir parcouru le roadmap , décrivant l'ordre de lecture et d'apprentissage des nouveaux fichiers.

Usage

```
./openstack-migrate.sh <vm_id> <target_host> [--dry-run]
```

- **<vm_id>** : Identifiant de la VM à migrer.
- **<target_host>** : Nom de l'hôte de destination.
- **--dry-run** (optionnel) : Permet de tester la faisabilité de la migration sans l'exécuter réellement.

Explication du script

1. Déclaration des variables

```
VM_ID=$1  
TARGET_HOST=$2  
DRY_RUN=false  
[ "$3" == "--dry-run" ] && DRY_RUN=true
```

- Récupération des arguments passés en ligne de commande.
- **VM_ID** contient l'ID de la VM.
- **TARGET_HOST** contient le nom de l'hôte cible.

- **DRY_RUN** est un booléen initialisé à **false**, qui devient **true** si l'option **--dry-run** est spécifiée.

2. Définition des fichiers de configuration

```
JSON_FILE="/etc/openstack/migration_rules.json"
CSV_FILE="/etc/node-clusters.csv"
LOG_FILE="/var/log/openstack_migration.log"
```

- **JSON_FILE** contient les règles de migration en JSON.
- **CSV_FILE** mappe les nœuds aux clusters et VLANs.
- **LOG_FILE** est le fichier où seront enregistrés les logs.

3. Fonction de journalisation

```
log() {
    echo "[$(date +%Y-%m-%d %H:%M:%S)] $1" | tee -a $LOG_FILE
}
```

- La fonction **log** ajoute un horodatage aux messages et les écrit à la fois sur la sortie standard et dans le fichier de logs.

4. Détermination du cluster source

```
get_source_cluster() {
    local vm_host=$(openstack server show $VM_ID -c
OS-EXT-SRV-ATTR:hypervisor_hostname -f value)
    awk -F, -v host="$vm_host" '$1 == host {print $2}' $CSV_FILE
}
```

- Extrait le nom de l'hyperviseur qui héberge la VM via OpenStack.
- Recherche dans le fichier CSV à quel cluster appartient cet hyperviseur.

5. Vérification de la connectivité réseau

```
check_network() {
    local source_vlan=$(awk -F, -v cluster="$SOURCE_CLUSTER" '$2 == cluster {print $3}'
$CSV_FILE)
    local target_vlan=$(awk -F, -v cluster="$TARGET_CLUSTER" '$2 == cluster {print $3}'
$CSV_FILE)
```

```

if [ "$source_vlan" == "$target_vlan" ]; then
    return 0
else
    openstack router list | grep -q "network:$source_vlan.*network:$target_vlan" && return
0
fi

log "ERREUR: Aucune route entre VLAN $source_vlan et $target_vlan"
return 1
}

```

- Vérifie si les VLANs des clusters source et cible sont identiques ou s'ils sont interconnectés via un routeur OpenStack.
- En cas d'absence de route, une erreur est journalisée.

6. Vérification de la compatibilité Xen

```

check_xen() {
    local source_node=$(awk -F, -v cluster="$SOURCE_CLUSTER" '$2 == cluster {print $1;
exit}' $CSV_FILE)
    local target_node=$(awk -F, -v cluster="$TARGET_CLUSTER" '$2 == cluster {print $1;
exit}' $CSV_FILE)

    ssh $source_node "xl info | grep -E 'xen_version|cc_compiler'" > /tmp/xen_source
    ssh $target_node "xl info | grep -E 'xen_version|cc_compiler'" > /tmp/xen_target

    if ! diff -q /tmp/xen_source /tmp/xen_target; then
        log "ERREUR: Incompatibilité Xen ($(cat /tmp/xen_source) vs $(cat /tmp/xen_target))"
        return 1
    fi
    return 0
}

```

- Vérifie la compatibilité des versions Xen et des compilateurs sur les hyperviseurs source et cible.
- Une incompatibilité bloque la migration.

7. Vérification des capacités CPU

```

check_cpu() {
    local required_flags=$(awk -F, -v cluster="$SOURCE_CLUSTER" '$2 == cluster {print $5}'
$CSV_FILE | tr ';' '\n')

```

```
local available_flags=$(awk -F, -v cluster="$TARGET_CLUSTER" '$2 == cluster {print $5}'
$CSV_FILE | tr ';' '\n')
```

```
for flag in $required_flags; do
    if ! grep -qx "$flag" <<< "$available_flags"; then
        log "ERREUR: Flag CPU manquant ($flag)"
        return 1
    fi
done
return 0
}
```

- Vérifie si le processeur de la machine cible supporte tous les flags CPU nécessaires.
- Une absence d'un flag requis empêche la migration.

8. Vérification des règles JSON

```
check_json() {
    local result=$(jq -r --arg src "$SOURCE_CLUSTER" --arg dst "$TARGET_CLUSTER" \
'[$src][$dst] // {"success": false, "info": "Combinaison non documentée"}' $JSON_FILE)

    success=$(jq -r '.success' <<< "$result")
    info=$(jq -r '.info' <<< "$result")

    [ "$success" == "true" ] && return 0
    log "ERREUR: Règle JSON bloque la migration - $info"
    return 1
}
```

- Vérifie si la migration est autorisée selon les règles JSON définies.

9. Exécution des vérifications et migration

```
SOURCE_CLUSTER=$(get_source_cluster)
TARGET_CLUSTER=$(awk -F, -v host="$TARGET_HOST" '$1 == host {print $2}'
$CSV_FILE)
```

```
log "Début migration $VM_ID de $SOURCE_CLUSTER vers $TARGET_CLUSTER"
```

```
checks_passed=true
check_network || checks_passed=false
check_xen || checks_passed=false
check_cpu || checks_passed=false
check_json || checks_passed=false
```

```
if $checks_passed; then
  if $DRY_RUN; then
    log "DRY RUN: Migration possible"
    exit 0
  fi

  if openstack server migrate --live $TARGET_HOST $VM_ID --wait; then
    log "Migration réussie"
    openstack server resize --confirm $VM_ID
    exit 0
  else
    log "Échec de la migration technique"
    exit 1
  fi
else
  log "Migration bloquée - Voir logs détaillés"
  exit 1
fi
```

- Exécute toutes les vérifications avant d'effectuer la migration.
- Si `--dry-run` est activé, il simule la migration sans l'exécuter.
- Si une vérification échoue, la migration est annulée.

.